AI SYSTEMS SCIENCE / AGENT SYSTEMS

32讲 · 零基础理论课程 • 第一部分：基本原理

02

为什么大语言模型需要外部记忆？

Why Do LLM Agents Need External Memory?

从上下文窗口和Token预算出发，逐步理解长上下文的能力边界、跨会话连续性与外部记忆的架构价值。

零基础入门 | 只讲理论 | 6张图 · 贯穿案例 · 思考题

COURSE ROADMAP

本讲学习地图

先沿“上下文→容量→跨会话→有效利用→外部记忆”的逻辑阅读，再用理论题检查是否能够独立推导。

00	本讲学习目标
01	1. 先从一个四周科研项目开始
02	2. 模型回答一次问题，究竟依靠什么信息？
03	3. 什么是上下文窗口？先把 Token 想清楚
04	4. 第一个困难：信息总量持续增长，但输入预算有限
05	5. 第二个困难：一轮对话结束，模型不会自动把新经历写进参数
06	6. 第三个困难：文本放进长上下文，也未必被稳定利用
07	7. 第四个困难：把所有历史反复发送，可能非常低效
08	8. 第五个困难：知识和决策会变化，长历史不能自动处理“过时”
09	9. 外部记忆究竟怎样解决这些问题？
10	10. 三条技术路线：全量上下文、摘要、外部检索
11	11. 把三种方法放进同一个案例中检验逻辑
12	12. 外部记忆何时不值得引入？
13	13. 用最少的符号建立一个系统模型
14	14. 八个常见误区：必须能说出错在哪里
15	15. 理论思考题：先独立推演，再看答案
16	16. 思考题参考答案与推导
17	17. 一页带走：本讲必须留下的知识模型
18	18. 原始研究文献与扩展阅读

贯穿的核心问题

当历史越来越长 → 哪些信息应该保留 → 如何进入当前上下文 → 如何避免旧决定误导行动

第02讲 为什么大语言模型需要外部记忆?

课程：《智能体记忆系统：从零基础到科研理解》（32讲理论课程）
单元：第一部分 · 智能体记忆系统的基本原理
讲义版本：v1.0 · 2026年10月17日
学习定位：零基础、理论优先；本讲不进行实验、不要求编程。
与上一讲的关系：第01讲建立了“观察—写入—存储—检索—使用—更新”的整体架构；本讲回答一个更基本的问题：为什么不能让大语言模型仅凭自己的参数和越来越长的上下文完成长期任务?

本讲学习目标

学习结束后，你应当能独立解释：①上下文窗口到底是什么、Token为何重要；②对话记录为何不等于模型真正记住；③长上下文为什么不等于稳定、持久、可更新的记忆；④窗口容量、信息可用性、时间连续性和成本四类问题的区别；⑤外部记忆如何在多次会话之间把必要信息带回当前决策；⑥什么时候外部记忆反而不需要或可能带来负面作用。

> **本讲的总问题**：如果今天的大语言模型能够处理很长的文本，为什么我们还要专门设计一个智能体记忆系统？答案不是“因为上下文太短”这么简单；即使窗口扩大，**过去的信息能否稳定留下、被正确找回、与新信息协调并影响决策**，仍是相互独立的问题。

1. 先从一个四周科研项目开始

想象一个科研智能体，每天帮助你推进“如何提高智能体长期记忆的可靠性”。这不是四个彼此无关的问答，而是一项持续演化的工作。

时间	真实工作事件	智能体未来真正需要知道什么
第1天	确定方向：研究长期记忆机制，不急于定具体算法	长期目标与判断原则
第6天	提出假设：把全部对话放进模型，就不需要独立记忆	一个尚未证实的候选想法
第15天	阅读论文后发现：相关内容位于长输入中间时，某些模型较难使用	重要证据，但必须保留任务与模型条件
第21天	因成本与错误风险，决定不把“全部历史原样拼接”当作唯一方案	已生效的新决定；先前想法不再直接适用
第30天	你问：“我们的记忆方案目前应该怎样设计？”	综合旧目标、新证据、最新决定后提出建议

只看第30天这句话，语言模型无法从空气中知道第21天的决定。即使语言表达能力很强，也可能重新建议第6天已经被否决的方案。

若第1至30天的全部消息都还在当前上下文中，它有可能直接找到这些信息。但随着项目变长，系统必须回答：输入是否装得下？是否能找到正确片段？什么内容已经失效？值得为所有历史反复付出多少计算代价？

若使用外部记忆，系统可以把第21天的决定和第15天的证据以可追溯的形式保存，到了第30天按需要检索，而不必默认把30天内所有文本都重新送入模型。但检索本身也可能失败，不能认为一加数据库就自动可靠。

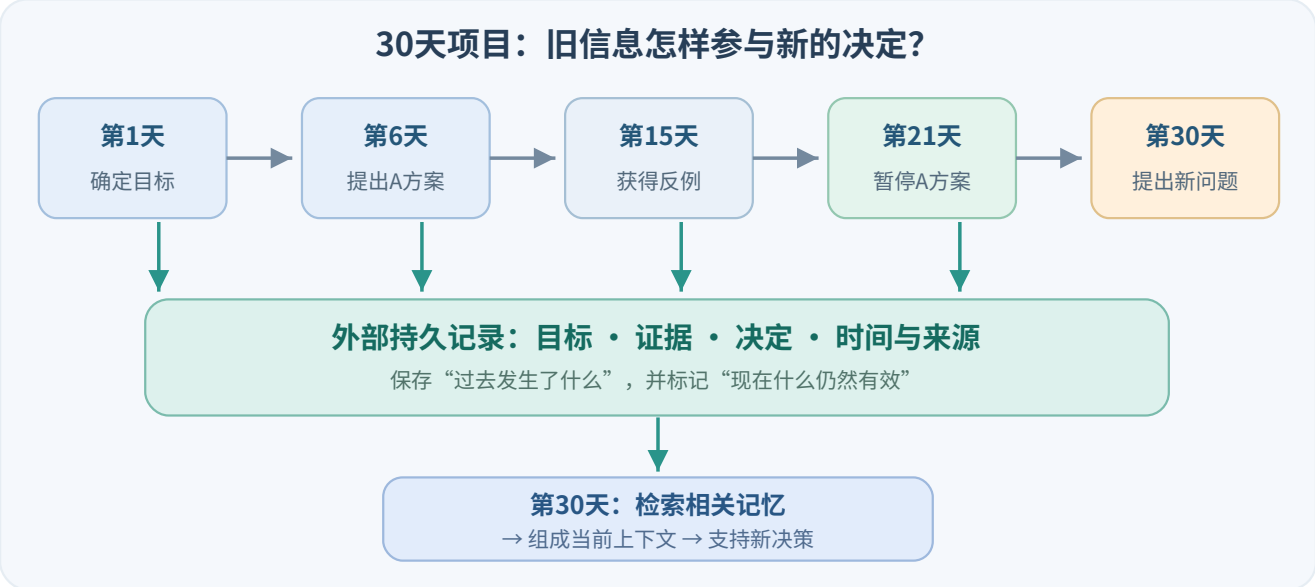


图1 · 教学示意：把多天的事件保存为可查状态，在新的任务到来时按需找回。

1.1 从故事抽象出四种不同的困难

它们常常同时出现，却不是同一个问题。

1. **容纳问题**：有多少信息能进入这一轮的模型输入？
2. **持久问题**：这一轮结束以后，重要信息在哪里，下一轮能否重新获得？
3. **利用问题**：即使信息已经进入输入，模型是否能正确识别、理解和使用？
4. **维护问题**：过期、被否决或互相矛盾的信息，应该怎样影响当前决定？

这四个问题形成理解本讲的主线。注意：只有第一个问题能被“单纯扩大上下文窗口”直接缓解；其他问题通常仍需系统层机制共同处理。

2. 模型回答一次问题，究竟依靠什么信息？

先澄清一个常见印象：人类与AI聊天时，看到的是一个连续对话框，似乎模型始终在里面“陪着我们”。但从一次典型的文本推理请求看，模型真正处理的是**这一轮被发送给它的输入序列**。

这条序列可以包含系统与开发者指令、当前用户问题、若干历史消息、工具返回内容、外部检索的记忆片段，以及其他模态转化后的输入。对话应用负责如何挑选与组合这些内容；模型并不天然直接访问产品数据库里的所有历史。

例如，一次简化后的输入可能是：

- 系统规则：你是科研助手，所有方案必须根据最新证据判断。
- 用户当前问题：我们下一步选哪条记忆路线?
- 最近消息：讨论了质量与成本的取舍。
- 系统检索记忆：第21天已经暂时否决“全历史原样拼接”。
- 工具摘要：第15天记录了一项长上下文使用位置可能影响表现的研究。

如果“第21天决定”没有被发送进去，或模型也没有其他方式通过工具访问它，模型便不能可靠地利用这条特定历史。模型可以给出符合常识的猜测，但**猜得像事实，不等于检索了事实**。

2.1 三个层次，不要混在一起

模型参数：预训练、微调等过程形成的权重，承载一般知识和能力；普通的一次用户对话通常不改权重。
当前上下文（Context）：本轮实际呈交给模型处理的信息，是当前计算可见的材料。
外部存储与记忆管理：保留在请求以外的历史记录、索引、文件和结构化状态，只有通过检索、读取等机制才进入后续任务。

三者可共存。你要判断一种架构有没有“长期记忆”，不能仅仅因为它会回答刚刚说过的话就下结论。那可能只说明相关话语还在上下文中。

2.2 一个必须区分的层次：聊天应用中的历史记录

聊天界面把第1天的消息保存在服务端，不代表第30天的模型请求必然携带第1天的全文。应用可以采取完整回放、滑动窗口、摘要、按需检索或其他方式。**“数据库里有聊天历史”和“当前模型看见聊天历史”是两个事实。**

也不要将“产品有跨会话记忆功能”与“模型参数被实时训练”混为一谈。产品级记忆往往由应用层与存储、检索和上下文组装共同实现，实际机制因产品而异。

3. 什么是上下文窗口？先把 Token 想清楚

上下文窗口（Context Window），在入门层面可以理解为：模型在一次请求的处理中可接纳或有效处理的输入与生成内容的长度范围，通常用 **Token** 表示。不同模型与接口对输入长度、输出长度、缓存、工具消息的计数方式可能不同，不能把一个产品宣传的窗口数字直接等同于全部可用的聊天历史容量。

Token（词元）是模型用于表示文本的基本离散单位，并不固定对应一个汉字、一个英文单词或一个字节。一个汉字可能对应一个或多个Token；英文常见词有时是一Token，也可能被拆分；标点、空格和代码同样消耗Token。因此，**十万Token不等于十万汉字**。精确数量取决于使用的Tokenizer和文本内容。

3.1 用一个完全虚构的预算例子理解窗口

假设某个教学用模型的**总有效上下文预算为16,000 Token**。这只是为了帮助理解的演算例子，不对应本课程声称的任何具体商业模型参数。

项目	占用 Token	解释
系统规则和角色信息	1,600	必须遵循的约束与角色
当前问题及近期必要对话	1,200	眼下究竟要完成什么
工具返回的论文/数据摘要	3,600	来自外部环境的新材料
本轮检索到的记忆	2,000	与当前问题直接有关的过去
为本轮回复预留的长度	1,600	不可把所有空间都给输入
可留给其他历史记录	**6,000**	通过计算得到的剩余预算
合计	**16,000**	教学演算用假设

计算关系：16,000 − (1,600 + 1,200 + 3,600 + 2,000 + 1,600) = **6,000**。



图2 · 数字均为教学假设，展示各类信息竞争同一可用预算；不对应任何具体产品参数。

看出关键了吗？即使模型号称有16,000 Token的可用空间，能够放进来的**旧对话历史并不是16,000 Token**。规则、当前问题、工具内容、记忆片段和回复空间都要占位置。在真实产品中，不同供应商的限制和记账方式可能更复杂。

3.2 三种长度不要混淆

- **标称窗口**：模型或接口宣称支持的最大范围。
- **实际送入的上下文长度**：应用这一轮真的组织了多少信息。
- **有效利用范围**：虽然送进去了，模型能否稳定抽取并综合信息。

这三个量没有必要相等。窗口很大但应用只发送最近几轮，系统仍会缺少旧信息；历史虽然全部发送，模型也可能遗漏中间的关键约束。

3.3 “窗口大小”不是记忆保留期限

窗口限制说的是**一次推理可见/处理的信息量**，不是“记忆最多保存30天”。一个外部数据库可以保存几年消息，而当前模型窗口很小；另一个没有外部持久记录的应用即使使用很大窗口，也可能在会话断开后无法自动接续过去。这是两个相互独立的设计维度。

4. 第一个困难：信息总量持续增长，但输入预算有限

现在把科研项目从30天延长到一年。每天的讨论、资料阅读、工具输出、摘要、修订意见都在增加，而一轮回答仍需在某个有限信息范围内计算。

如果完全依赖“从项目开始到现在的全部历史原样拼接”，历史总量可能超过可用窗口。此时应用不得不做选择：**截去一些历史、压缩历史、分块多次处理，或按需从外部记录中选择内容**。

4.1 最直观的方法：只保留最近若干轮

所谓“滑动窗口”可以这样理解：模型只看最近的一段历史，新的消息进来，最旧的消息被移出当前输入。它能保持近期话题连贯，却可能漏掉数周前一个仍然有效的目标或限制。

例如，第1天你说“暂时只研究记忆可靠性，不追求扩大记忆数量”；第30天讨论方案时，这句话可能早已被挤出窗口。模型在第30天的回答不再受它约束。

这里所谓“移出窗口”，**不意味着服务端历史记录一定被物理删除**。可能只是没有被放进这一轮模型请求。是否保留原始记录，取决于应用的数据设计。

4.2 第二种办法：先给历史做摘要

可以把10万Token的对话压缩成1,500Token的摘要，例如：“当前研究聚焦智能体记忆可靠性；此前否决了只扩充记忆数量的方案。” 摘要降低了输入需求，但也会丢失细节，例如“否决的适用范围”“原始证据位置”“谁作出的决定”。

压缩不是无损存储。 摘要好比把十页会议纪要写成半页概要：重要结论可能保留，但无法保证每个定义、数字、否定条件和来源都完整留下。若摘要出现偏差，后续继续从摘要生成摘要还可能积累误差。

4.3 第三种办法：记录放在外部，需要时再取

外部记忆把“永久保存多少”和“这一轮取回多少”拆成两项独立决定：可以保留大量历史，只提取本轮真正相关的少量信息。这是一个典型的**存储容量与工作上下文解耦**设计；MemGPT借鉴了操作系统的分层记忆与虚拟上下文管理思想。[3]

这不等于免费午餐。系统必须处理检索召回不足、错误排序、信息过时、权限保护、写入/维护成本等问题。后面的课程会专门研究。

5. 第二个困难：一轮对话结束，模型不会自动把新经历写进参数

很多初学者会问：“我刚才告诉模型的事情，它明明回答得出来，怎么到了下次又不知道了？”

因为在典型部署中，**普通模型推理不会因一条聊天消息就修改底层模型参数**。当前会话出现过的信息可能只存在于请求上下文与应用的历史数据库里。

5.1 一个具体场景：从“眼前知道”到“长期保存”

第1天你说：“我们的课程只有理论，不安排任何编程实验。” 当前上下文含有这句话，模型当然可能按此要求生成讲义。

几周后你重新建立一个全新请求，只发送：“继续下一讲。” 如果没有带入课程大纲和“暂不实验”的历史约束，模型很可能无法确定这句话到底指哪门课程，也不知道你的要求。即使它猜测出一个课程方向，也不说明它恢复了原始约束。

要实现真正的连续工作，应用层需要某种机制把“课程名称、进度、约束、讲义链接”等状态保留下来，并在后续对话中重新提供给模型。这可以是结构化任务表、对话摘要、知识库、专门记忆系统等，不一定都要叫“向量数据库”。

5.2 训练更新与记忆写入不是一回事

修改模型参数一般涉及训练或参数更新流程，改变底座模型的内部权重；**写入外部记忆**通常是保存一条可供后续访问的数据。前者可能影响模型很多任务，后者一般通过特定索引或工具影响访问该记录的任务。

如果只想让科研助手记住“第21天否决了A方案”，训练整个大模型通常不是必要或合理的第一选择。更直接的方法是保留一条有时间、来源、状态的外部记录，并在相关任务时读出它。

5.3 但“外部记忆”也不是唯一持续状态形式

有些系统可以保存更复杂的运行状态，例如任务状态机、数据库记录、文件版本、代理执行轨迹；有些模型采用特殊的循环状态或可写内部机制。**本课关注常见LLM智能体的外部持久记忆与管理机制，但不把所有实现一律缩减为“文本进向量库”。**

6. 第三个困难：文本放进长上下文，也未必被稳定利用

这是最容易被“更大窗口”掩盖的一点：**装得下**和**用得好**属于两个不同能力。

6.1 一条信息在开头、中间、结尾，为什么可能有不同结果？

想象给模型一份很长的会议记录，里面只有一处说明“最终采用B方案”。若这句话在记录最开头，模型可能较容易抓到；在最后，也可能较容易；但当它被大量无关段落包围，处在中间位置时，部分模型就可能遗漏或错误利用。

Liu等人的 Lost in the Middle（TACL 2024）在多文档问答与键值检索实验中报告：所测试模型的表现受相关信息位置影响，常呈现开头和结尾较好、中间较差的现象。[1] **这是一组实验发现，不是宣称所有模型在所有任务里必然表现为同一条曲线。**

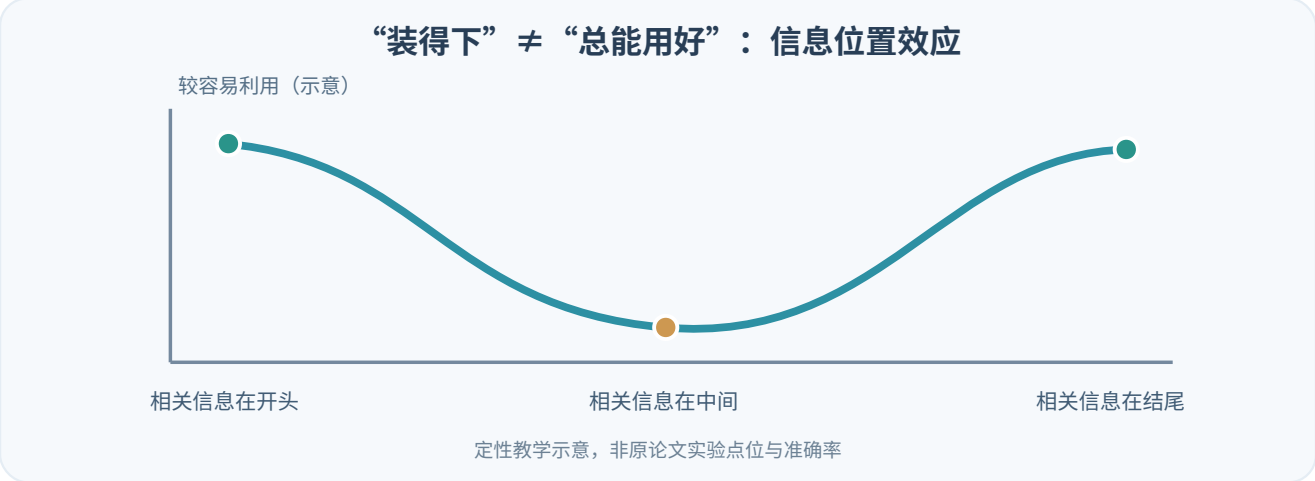


图3 · 用曲线帮助理解Liu等人报告的某些实验中的位置效应，不代表所有模型或任务。

这里值得推导：如果第21天那条“停止A方案”的决定刚好处于一年历史的中部，即使没有超过窗口限制，也可能被干扰信息遮蔽。于是“用更长上下文”只能缓解输入不足，未必解决信息检索与使用的可靠性问题。

6.2 为什么不能只用“针在干草堆里（Needle in a Haystack）”做判断？

“干草堆找针”常测试模型能否从长文本里找出一个特定字符串或事实。这个任务有价值，但科研任务通常更复杂：需要把第1天的目标、第15天的证据、第21天的新决定组合起来，有时还要发现两条记忆互相冲突。

RULER研究指出，简单的单一事实查找不足以全面反映长上下文理解；它加入了更复杂的查找、多跳追踪和聚合任务，并观察到模型随长度增加而产生的性能下降。[4] 因此“能找出那句话”不等于“能理解整个项目现在应该怎么做”。

6.3 “正确检索”也不等于“正确行动”

假设外部系统检索出三条记忆：

1. 第6天：提出A方案，尚待验证。

2. 第15天：观察到若干与A方案预期不符的结果。
3. 第21天：决定暂时不继续A方案。

三条都是真实历史，但它们在当前决策中的地位不同。如果模型仅依据“出现次数最多的A方案”给出建议，仍会出错。正确系统不仅要把信息找回来，还需要识别**最新性、有效性、来源可信度及结论状态**。

LongMemEval（ICLR 2025）专门评估跨会话信息提取、多次会话推理、时间推理、知识更新和应当拒答的情况，其设计本身就体现了“长期记忆能力不等于单次文本查找”的观点。[2]

7. 第四个困难：把所有历史反复发送，可能非常低效

即使未来某个模型支持更大的上下文，也必须考虑**每次任务实际需要什么、处理这些信息消耗多少资源**。

7.1 为什么长输入会增加处理开销?

把两页材料与两百页材料都放进一次请求，模型需要处理的内容不同。输入Token越多，通常涉及更多数据传输、模型计算和可能的费用。具体费用与延迟取决于模型、硬件、并发、缓存、计费方式和优化实现，不能只根据窗口大小推断。

对传统**稠密全注意力（dense full attention）**而言，在长度为n的序列上，注意力分数相关的计算量通常随n^2增长。于是把n翻倍，相关成对交互的理论数量会变为原来的四倍。

> **注意这个限定很重要**：n^2讨论的是特定注意力计算结构，并不等于整个模型的实际端到端延迟一定四倍，更不能推导所有API费用都会四倍。FlashAttention、KV缓存、稀疏或分块算法等实现会改变实际内存和运算特性。

7.2 用“每天查一个结论”来理解重复成本

假设一份项目历史已经达到10万Token，而第30天只想问“第21天为何停止A方案？”

方案一：全部拼接。系统把10万Token都交给模型，虽然问题涉及的可能只是其中几百Token，但也把大量无关历史当作输入。
方案二：按需检索。系统先从长期存储中找到第15天的证据与第21天的决定，组织约2,000Token的证据再回答。它减少送入模型的材料，但多出检索工作，且可能漏掉关键来源。

这个例子中的“10万”和“2,000”只是为了呈现两种设计思路，并不构成真实产品的性能比较结果。只有控制检索质量、模型能力、数据规模和计费方式之后，才能判断哪种方案真正更优。

7.3 成本还包括“维护”而非仅仅是Token

外部记忆需要写入、更新、索引、备份、权限控制，有时还要额外调用模型总结历史。对于一次性的小问题，这些工作可能完全不划算。记忆系统的目的不是越复杂越先进，而是让长期工作在**质量、成本、延迟和可维护性**之间有合理平衡。

8. 第五个困难：知识和决策会变化，长历史不能自动处理“过时”

接下来考虑一个与容量无关的问题。第6天你提出“采用A方案”，第21天你明确说“暂停A方案”。这两句话都应该能从历史里找到，但它们不能都当作当前生效指令。

8.1 历史事实和当前有效状态不是同一件事

“第6天曾提出A方案”是**历史事件事实**；“目前应该采用A方案”是**当前决策状态**。第二句话可能因新证据而失效，但第一句话仍然真实。一个可靠的记忆系统既要保存历史，也要能给出最新状态。

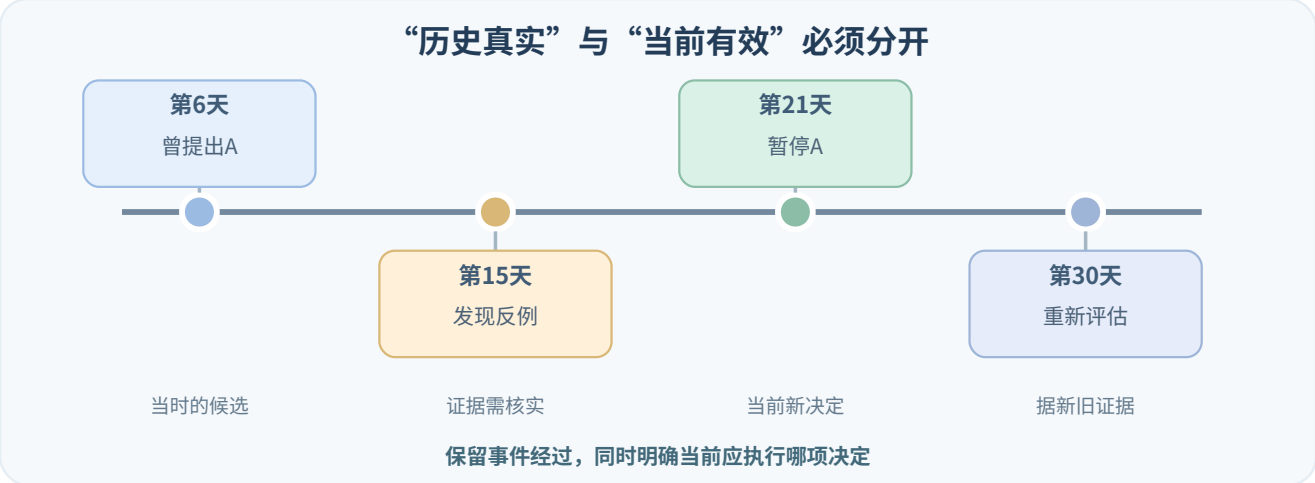


图4 · 旧决定不必从历史里抹去，但不得将其误认为仍然有效的行动指令。

如果系统把时间顺序完全丢掉，就可能让旧结论覆盖新决定；如果粗暴删除所有旧记录，未来要解释“为什么改变方向”时又会丢失推理链和责任依据。

8.2 摘要、检索和长上下文都不能独自保证冲突处理正确

- 长上下文可以同时包含新旧结论，但模型仍需识别其时间与效力。
- 摘要可以保留“当前采用B”，但可能删除导致变更的证据和条件。
- 检索可以召回某条决定，却可能只找回“曾采用A”而漏掉后来的撤销。

因此长期记忆至少需要某种**版本、时间、来源、有效性或冲突处理机制**。具体怎样设计我们在第31—35讲深入学习，本讲先建立问题意识。

8.3 另一种维护难点：信息本身可能根本不可靠

例如工具返回一段自动生成的“论文摘要”，其中某个结果被写错。若系统把它直接永久写入，再在几个月后当作权威事实使用，那么记忆机制会把最初的一次错误反复传播。

这说明记忆不仅是存储工程，还牵涉证据质量。**记忆能够放大经验价值，也可能放大历史错误。**

9. 外部记忆究竟怎样解决这些问题?

可以把外部记忆看成一套“在过去的信息与当前计算之间建立受控连接”的机制。它的关键不是把模型输入变得无限大，而是让信息能**在需要时重新出现**。

9.1 从一条消息开始，按步骤推演

以第21天“停止A方案”为例：

第1步：形成候选记忆。 新决定出现时，系统识别它属于会影响未来的关键事项。
第2步：结构化写入。 保存内容“当前暂停A方案”，时间“第21天”，理由“成本与错误风险”，来源“用户决定”。
第3步：持久化存储。 记录进入数据库、文件或其他长期载体，并建立可查询入口。
第4步：第30天发起检索。 用户询问方案方向，系统判断需要历史决策。
第5步：召回并检查冲突。 取回第21天的当前决定，并可同时关联第6天的原始假设和第15天的证据。
第6步：交给模型使用。 以少量相关信息构造当前上下文。
第7步：行动后写回。 如果第30天出现新的结论，系统判断是否更新记忆，构成下一轮循环。

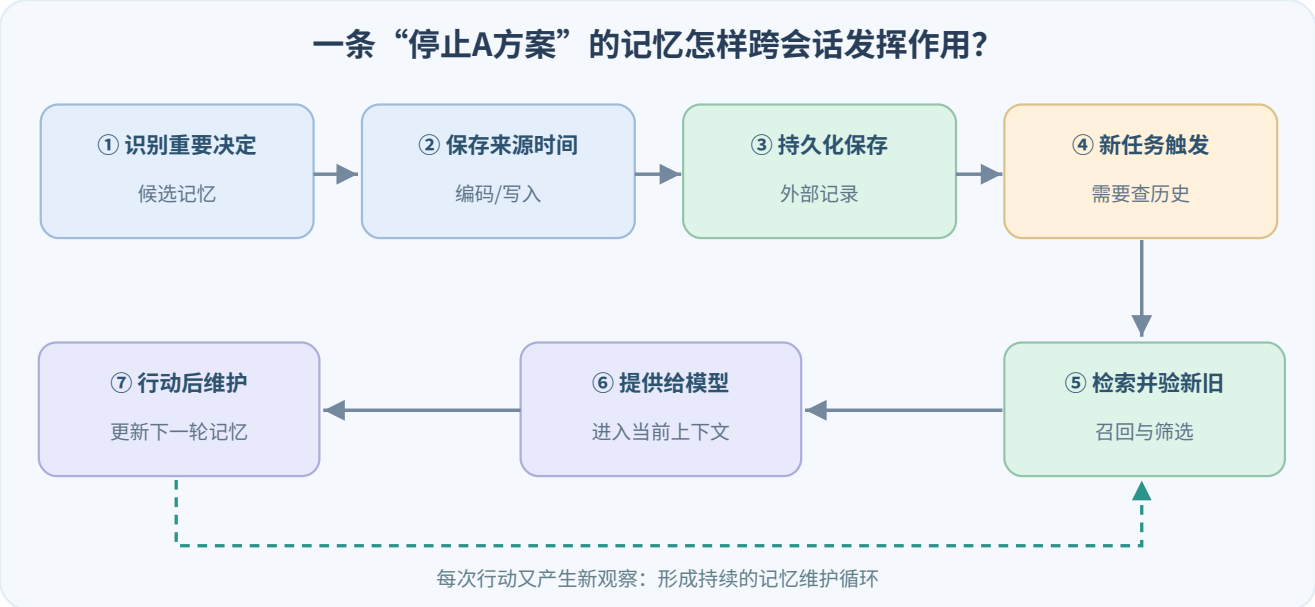


图5 · 简化的写入、保存、检索、利用和维护过程；实现中可由不同控制器或工具承担。

注意：这七步是帮助建立理解的教学流程，不代表每一个产品都逐步执行，有的阶段可合并、跳过或交由控制器管理。

9.2 外部记忆缓解什么？不能自动保证什么？

目标	外部记忆能提供的机制	不能自动保证的结果
跨会话保留	把信息持久化保存	记录一定被写入或下次找到
大量历史	当前只取回小部分相关内容	被取回的就是全部重要信息
降低重复输入	避免每轮携带完整历史	总费用必定更低
追溯旧决定	保存事件时间、来源与链接	模型必定服从最新决定
处理信息变化	修订、标记失效、记录版本	冲突总能自动解决

一个记忆系统是否真的有用，最终还必须看**任务决策是否改善**。仅仅让它能保存100万条记录，不足以证明它记忆“好”。

10. 三条技术路线：全量上下文、摘要、外部检索

现在可以把最常见的三种路线放到同一张设计空间图中。它们并非绝对互斥；成熟系统往往组合使用。



图6 · 三种路线可互相组合；比较属于架构机制的定性讨论，不是测得的排名。

10.1 全历史回放：简单、直接，但扩展性有边界

做法：把所有相关历史原样放进当前输入。优点是保留原文细节、没有单独检索漏召回问题；缺点是容量、冗余、干扰与重复处理成本。适合历史相对短、细节重要、模型有足够有效窗口的任务。

特别提醒：全历史保留并非错误方案。与外部检索相比，当待处理历史不长而查询需要跨越多个分散细节时，完整上下文可能更可靠；必须根据任务和模型比较，而不是先入为主排斥。

10.2 滚动摘要：输入轻量，但丢失信息可能不可逆

做法：每过若干轮，把历史压缩成摘要，再用摘要加近期对话继续。优点是便宜、简单，尤其适合维护稳定目标和简单状态；缺点是压缩时若删去了关键限定条件，后来就无法从摘要本身重建原句。

长期项目中，更合理的做法可能是让摘要担任“导航卡”，而不把它当作唯一证据源；需要时还能访问原始记录或更完整的材料。

10.3 外部记忆与按需检索：可扩展、可维护，但链路变长

做法：保存历史记录、提取重要事实或事件，必要时检索并组织给模型。优势是存储与当前上下文解耦，也便于维护来源、时间与状态；缺点是写入、检索和更新都会出错，工程复杂度增加。

例如LongMemEval把系统性设计问题分为索引、检索与阅读阶段，说明最终效果来自多个协作环节，而不是“用了某种数据库”这一项。[2]

10.4 比较时不要问“谁先进”，要问“谁在这个任务里更合适”

比较维度	全历史回放	滚动摘要	外部记忆检索
保存原文细节	有条件地完整	通常受压缩损失	可留原文并按需取回
历史规模扩展	受窗口与成本约束	容易压缩变短	存储易扩展，检索仍需设计
关键事实漏失风险	模型在长输入中漏用	摘要阶段删掉	写入或检索阶段漏掉
时间与版本维护	模型需要自行辨析	可在摘要中维护	可显式记录、更新与查询
实现难度	通常较低	中等	取决于架构，可能更高
典型适用	短至中等的完整任务	任务总览与连续对话	长期、多会话、可追溯任务

这是**机制层面的定性比较**，不是某篇论文对所有方法的量化排名。正式判断优劣必须使用相同模型、任务、可见信息、资源口径和评价方式。

11. 把三种方法放进同一个案例中检验逻辑

第30天，你问“为什么第21天我们放弃了A方案，现在该怎么继续？”

全历史回放：若所有必要消息都装得下、模型能准确处理，模型可以查看第6、15、21天原文，给出解释。问题是随着历史变长，时间与成本负担增加，且可能受到不相关内容干扰。

滚动摘要：若摘要保留“第21天因成本与错误风险暂停A方案”，就可能快速回答；但若摘要只留“研究A方案并讨论成本”，那关键的否决决定已丢失。这是**摘要压缩损失**，不是模型推理必然失败。

外部记忆：检索器理想情况下找到第21天的决定，并附带第15天的证据；但如果只根据关键词“A方案”检索，返回第6天的旧建议，也会出现**检索偏差**。若记忆库存在但从未被调用，则属**触发/调度失败**。

由此可见，三种方案的关键失效点不同。科研时必须沿数据流寻找最先发生偏差的环节，而不能简单把“回答错误”都归因于“大模型记性不好”。

12. 外部记忆何时不值得引入?

不是每个问答系统都需要长期记忆。以下情况可能更适合简单方案。

情况A：一次性问题。只问“解释TCP三次握手”，外部保存个人历史往往无助于完成任务。
情况B：一个短文档内即可回答。所需资料能完整而可靠地放入当前上下文时，引入独立记忆库可能增加索引和检索错误。
情况C：隐私约束较强。即使记忆有便利性，某些信息也不应自动长期保存；需要有授权、用途边界与删除控制。
情况D：事实变化非常频繁且必须依赖权威实时源。保存的旧答案不应替代当前查询，例如实时航班状态或实时账户余额。

更重要的原则是：**外部记忆应该服务任务，而不应该反过来强迫任务适应记忆系统。**

12.1 一个容易被忽略的负面效应：错误记忆可以使结果更差

假设模型本来能根据当前论文和问题提出合理建议，却被记忆系统塞进一条已经过时的“建议永久停止所有实验”记录。模型可能受到误导，产生比没有记忆时更差的决策。

因此，是否要检索、是否信任记忆、记忆是否过时，以及当前证据是否足以覆盖历史经验，都是“更好记忆”不可回避的部分。**有用的系统未必存得最多，而可能是只在恰当时候提供恰当的过去。**

13. 用最少的符号建立一个系统模型

此处公式只是把概念连起来，不要求你学习数学推导。

13.1 当前上下文预算

记一次回答可以使用的总预算为 **W**（window budget）。将本轮需要的系统规则记为P、当前问题记为Q、历史片段记为H、工具信息记为T、检索记忆记为M、预留回复空间记为R，则可写成：

> **P + Q + H + T + M + R ≤ W**

这表示一件非常直观的事：多个内容来源都在争用一个有限的可用信息预算。这里不讨论不同提供商计费、缓存与输出长度约束的细节；公式只是**教学抽象**。

13.2 外部记忆怎样改变问题

不使用外部记忆时，系统可能依赖越来越大的H，即尝试把大量历史直接放入当前输入。有外部记忆后，系统把部分历史保留在外部存储S中，通过检索得到更小的M：

> **长期历史 S → 检索/筛选 → 相关记忆 M → 当前上下文 → 决策**

这里的S可以很大，而M受当前预算与检索策略约束。**S大并不自动代表M好，也不自动代表决策好。**

13.3 记忆质量是一条链，而不是一个容量值

可以用一个因果链描述：

> **真实事件 → 正确写入 → 可查的表示 → 正确检索 → 正确解释 → 合理行动**

如果第21天的决定未写入，后面再强的检索也无济于事；如果写入正确却返回了错误版本，后面的推理就可能被污染；如果全部正确返回，但模型忽略了“已撤销”，决策仍失败。

这条链有助于未来阅读科研论文：检查作者改进的是哪个环节，以及最终成功率提升能不能归因于那个环节。

14. 八个常见误区：必须能说出错在哪里

常见说法	为什么不准确	应如何理解
“模型有很大上下文，就不需要记忆了”	长输入无法自动解决跨会话持久、更新和可靠使用	长上下文与外部记忆是可组合的技术
“聊天记录存在服务器，模型就全都知道”	模型不一定接收到完整历史	应区分存储状态与本轮可见输入
“我刚告诉它一件事，它就训练进参数了”	普通推理不等于权重更新	信息通常先存在上下文或外部系统
“100K Token就是10万个汉字”	Token不是固定长度汉字单位	实际数量取决于Tokenizer

常见说法	为什么不准确	应如何理解
“能找出一处字符串就能理解长期项目”	真实任务还涉及时间、关联与冲突	单点查找只测某一种能力
“把旧记录原样保留永远最安全”	旧事实可能过时，噪声影响判断	历史事实与现行结论应区分
“向量库越大，记忆一定越好”	规模不保证正确写入、召回或使用	评价应最终落到任务与成本
“外部记忆一定比全历史回放省钱且准确”	多了写入、检索、维护成本及错误链路	方法选择必须依赖任务场景

15. 理论思考题：先独立推演，再看答案

题1｜概念辨析 你在一个会话里说“今天研究记忆机制”，模型立刻回答“好的”。能否据此证明它已有跨会话长期记忆？请说出理由。

题2｜窗口预算 假设W=16,000，其中P=1,000，Q=2,000，T=4,000，M=3,000，R=2,000。还能给原始历史H留多少Token？如果历史已经有30,000 Token，至少可以考虑哪三种处理路线？

题3｜定位根因 第21天你否决A方案，第30天智能体又建议A。列出至少四种不同的失败可能，并说明为什么只看最后输出无法确定根因。

题4｜长上下文 某模型能准确从80,000Token里找出“预算上限是100元”，是否证明它能够在长期项目中正确处理“预算上限已从100元改为50元”？

题5｜摘要损失 原话是“在当前数据集、当前模型和初步实验条件下，增加记忆条数未带来明显提升”，摘要却写成“增加记忆条数没有用”。这个摘要在哪个环节出了什么问题？

题6｜设计取舍 有一个仅持续两轮、总共3,000Token的任务，另一个持续半年、有多个互相矛盾的历史决定。哪一个更值得引入可更新的外部记忆？为什么？

题7｜因果关系 已经在数据库保存第21天的决定，但第30天仍然答错。这是否意味着“存储层一定发生故障”？试着提出一个最小诊断顺序。

题8｜主动质疑 有人说：“未来模型的窗口总有一天会无限大，所以Agent Memory Systems是一个即将消失的领域。” 你应如何用容量、持久性、有效利用、动态维护和系统资源五个方面回应？

16. 思考题参考答案与推导

答案1｜不能证明

模型刚刚看到这句话，完全可以依据当前上下文回复，不需要外部存储，更不需要跨会话检索。要判断长期记忆，应检查新会话中是否存在可追溯的历史状态，并区分是不是应用自动带入了历史。**从一次“回答正确”无法反推底层机制。**

答案2｜剩余4,000 Token

计算：16,000−(1,000+2,000+4,000+3,000+2,000)=**4,000**。如果历史有30,000 Token，无法原样全部放进4,000预算。可以考虑：①按时间截断或保留近期片段；②做摘要压缩；③用外部存储保存历史并按需检索；④分段、多轮处理。需要根据内容损失、检索质量与成本选择。

答案3｜至少四条失效路径

其一，**未写入**：第21天的否决决定根本没有变成可用记录；其二，**表示失真**：写成了“讨论过A方案”，丢失“否决”这一动作；其三，**检索偏差**：只找到第6天的旧提议；其四，**时间判断错误**：找到了第6与21天记录，却没有识别最新决定；其五，**决策使用失败**：模型知道已否决，却仍然推荐旧方案。只有查看写入、存储、检索与最终上下文，才能定位。

答案4｜不能证明

找到一个静态字符串只检验了一类长上下文抽取能力。题目要求判断两个不同时点的预算上限哪个当前有效，还涉及时间推理、更新和冲突处理。LongMemEval把时间推理与知识更新单独设为测试能力，就是因为这些问题不能由单一“找针”完全覆盖。[2]

答案5｜摘要过度概括

原话包含“当前数据集、当前模型、初步条件”“未观察到明显提升”等限制，摘要将条件性观察扩大为无条件断言。错误发生在**记忆形成/摘要编码**阶段；即使以后检索非常准确，也只能准确取回一条已经失真的记忆。因此应保留范围、来源、证据状态。

答案6｜后一个更值得引入

短任务历史可完整处理，独立持久存储的维护成本可能超过收益。半年项目涉及多会话、过期决定、证据变更和可追溯性，外部可更新的记录更有实际价值。但仍不意味着必须引入向量数据库：结构化任务表、文件日志等也可能满足需求。

答案7｜不一定是存储故障

正确的最小排查路线：①确认历史决定原始文本是否被正确采集；②核对数据库记录是否存在且有正确时间/状态；③检查第30天是否触发历史检索；④检查检索结果是否包含最新决定；⑤查看这些结果是否进入模型

实际可见上下文；⑥检查模型是否依据正确结论行动。错误在任一位置都可能导致同样输出，故不可直接归咎于存储层。

答案8｜“无限窗口”不能自动消除全部系统问题

即使暂时假设未来存在足够大的上下文，**容量**限制可弱化；但跨会话状态是否**持久**存在仍要设计，模型对海量材料的**有效利用**不自动保证，旧决定如何被**更新和冲突解决**是额外能力，反复处理所有历史的**算力、延迟和费用**也需要权衡。并且外部权限、保留期限与删除等治理问题不能仅靠扩大窗口解决。故可以认为长上下文会改变记忆架构的最佳选择，却不能简单推出记忆研究失去意义。

17. 一页带走：本讲必须留下的知识模型

请尝试不看前文，独立解释以下逻辑。

第一步：模型一次只能依据实际可见的信息和自己的已有能力进行计算。 对话界面里还有旧消息，并不必然意味着本轮模型看到了它们。

第二步：上下文窗口是有限预算，Token不是固定数量的汉字。 系统规则、当前问题、工具返回、历史与输出预留都会占据预算。

第三步：长上下文只解决“可能装进去”的一部分问题。 信息位置、干扰、跨记录关联、时间更新和错误历史仍可能影响结果。相关论文报告了具体的长上下文使用困难。[1][4]

第四步：外部记忆把“长期保存”与“本轮计算”分开。 大量历史留在外部，依据当前任务选择一部分进入上下文；这是一种系统架构取舍，而非必然最优解。

第五步：外部记忆是一条可靠性链。 保存正确、检索正确、解释正确、行动正确，缺一不可。研究者应该关注整条链，而不只关注存储规模。

> **最重要的一句话**：大语言模型需要外部记忆，不只是因为“能看的字数有限”，更因为智能体的长期工作要求信息能够**跨时间保留、按需取回、识别新旧状态，并真正影响未来行动**。

与下一讲的衔接：第03讲《记忆、上下文、聊天记录与知识库有什么区别？》将把本讲出现的容易混淆的概念，通过更严格的定义、对照表与系统架构例子彻底区分。

18. 原始研究文献与扩展阅读

本讲文字中的研究结论对应下列可核实的原始来源。为便于零基础学习，推荐先读本讲，再根据兴趣逐步阅读论文；不需要现在立刻通读英文论文。

[1] Liu, N. F., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. **Transactions of the Association for Computational Linguistics**, 12, 157–173. DOI: 10.1162/tacl_a_00638. https://aclanthology.org/2024.tacl-1.9/ 。**与本讲关系**：为什么相关信息的位置会影响长上下文利用；注意论文的模型与任务范围。

[2] Wu, D., et al. (2025). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. **ICLR 2025**. https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html 。**与本讲关系**：多会话、时间推理、知识更新等记忆能力，以及索引—检索—阅读三个阶段。

[3] Packer, C., et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. https://arxiv.org/abs/2310.08560 。**与本讲关系**：通过分层与虚拟上下文管理处理有限窗口；这里引用原始预印本，不把它误写为某一指定期刊论文。

[4] Hsieh, C.-P., et al. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654. https://arxiv.org/abs/2404.06654 。**与本讲关系**：单一事实抽取测试为什么不能覆盖复杂的长上下文能力。

[5] Bai, Y., et al. (2024). LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. **ACL 2024**, 3119–3137. DOI: 10.18653/v1/2024.acl-long.172. https://aclanthology.org/2024.acl-long.172/ 。**与本讲关系**：长上下文需要用多任务真实能力检验。

阅读提示：本讲没有把研究中的特定准确率或图线复制成通用规律；所有带示意性质的数字与架构图仅用于解释概念，不是来自原论文的实验数据。